

Postal MS

Postal Service Management System

CST2550 • Software Development

The Leftovers

- Durga Prasad Avala (M01038074) - Team Leader & Developer
 - Meet Sanjay Mande (M01000862) – Secretary
 - Tapan Keyur Bhai Patel (M01050963) - Developer
 - Forhad Hossain (M01044942) - Tester
-

1. Introduction

PostalMS is a desktop postal service management system built in C# WinForms as part of our CST2550 coursework at Middlesex University. The application allows users to send parcels, track deliveries, order stamps, find nearby branches, request refunds and interact with an offline AI assistant. The system is backed by a Microsoft SQL Server 2022 database and uses three custom-built data structures at its core.

The three data structures -- a Hash Table, a Binary Search Tree and a Queue -- were all designed and implemented from scratch without using any standard template library collections, as required by the brief. Each structure was selected based on the specific performance requirements of the part of the application it supports (Cormen et al., 2022).

This report covers the system design and key decisions made during development, the design and analysis of the three data structures with pseudocode, the testing approach and results, and a reflection on what went well and what we would do differently.

2. System Design

2.1 Architecture

- **Presentation layer** C# WinForms. Each page is a UserControl and MainForm swaps them in and out.
- **Data access layer** A single DatabaseHelper class handles all SQL Server communication through stored procedures, keeping SQL out of the application logic.
- **Data structure layer** Custom Hash Table, BST and Queue sit in their own namespace and are used by the application to cache and process parcel data efficiently.

2.2 Database

- **Users table** Stores name, email, phone, address, postcode, city, role and password.
- **Parcel table** Stores all parcel records including sender and receiver details, weight, size, service type, destination and current status.
- **Delivery table** Tracks each delivery attempt with driver name, route, attempt count and outcome.
- **Stored procedures** All database operations use stored procedures following the principle of separation of concerns (Microsoft, 2024a).

2.3 Key design decisions

- **Stamps as a standalone page** Originally stamps were part of the parcels page. We rebuilt them as a completely separate section with browse, order and track tabs. The ordering flow works like a food delivery app with two cards: deliver or collect.
- **Pay online vs pay in store** Pay In Store is only available with Click and Collect. Selecting it with delivery shows a validation message.
- **Missed pickup rescheduling** If a collection slot is missed the order is held 48 hours then automatically rescheduled at the same location.
- **Gmail-only validation** All email addresses must end in @gmail.com. Enforced across all four forms in the application.
- **City-based Find Us** The Find Us page auto-detects the user's registered city and shows local PostalMS locations first. 76 locations across 7 UK cities.

3. Custom Data Structures

All three data structures were designed and implemented by the team from scratch. No List, Dictionary, SortedDictionary or Queue from the C# standard library was used anywhere in the application (Sedgewick and Wayne, 2011).

3.1 Hash Table

- **Purpose** Fast parcel lookup by tracking ID. The Hash Table converts a tracking ID into an array index using a hash function and jumps directly to that slot, giving $O(1)$ average lookup regardless of how many parcels exist.
- **Hash function** Polynomial rolling hash -- each character in the tracking ID is multiplied by 31 (a prime chosen to reduce collisions) accumulated and taken modulo the table size (Knuth, 1998).
- **Collision handling** Separate chaining -- each slot holds a linked list so multiple keys can share the same index without overwriting each other.
- **Auto-resize** When the load factor exceeds 0.75 the table doubles in size and re-inserts all items. This keeps collision chains short and maintains $O(1)$ average performance.
- **Complexity** $O(1)$ average for add, search and delete. Worst case $O(n)$ only if all keys hash to the same slot.

```
FUNCTION Add(key, value):  
    index = HashFunction(key)  
    walk chain at table[index]
```

```

IF key exists: update value and RETURN
ELSE: create new node, insert at front of chain
IF load factor > 0.75: CALL Resize()

```

3.2 Binary Search Tree

- **Purpose** Sorted parcel display. The BST maintains an ordering property so in-order traversal always returns parcels in ascending tracking ID order without any extra sorting step (Cormen et al., 2022).
- **How it works** Every left child has a smaller key, every right child has a larger key. Insert and search eliminate half the remaining tree at each step.
- **Complexity** $O(\log n)$ average for insert and search. Worst case $O(n)$ if items are inserted in already sorted order creating a degenerate tree.

```

FUNCTION InsertRecursive(node, key, value):
    IF node is null: RETURN new BSTNode(key, value)
    IF key < node.key: node.left = InsertRecursive(node.left, key, value)
    IF key > node.key: node.right = InsertRecursive(node.right, key, value)
    ELSE: node.value = value
    RETURN node

```

3.3 Queue

- **Purpose** FIFO delivery processing. Deliveries are assigned and processed in the order they are created, ensuring fair ordering.
- **Implementation** Singly linked list with front and rear pointers (Goodrich et al., 2013). Direct access to both ends means no traversal needed for enqueue or dequeue.
- **Complexity** $O(1)$ for enqueue, dequeue and peek with no worst case degradation.

3.4 Time complexity summary

Operation	Best	Average	Worst	Notes
Hash Table Add / Search / Delete	$O(1)$	$O(1)$	$O(n)$	Worst case rarely occurs
Hash Table Resize	$O(n)$	$O(n)$	$O(n)$	Amortised $O(1)$ per insert
BST Insert / Search	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$ only with sorted input
BST In-Order Traversal	$O(n)$	$O(n)$	$O(n)$	Visits every node once
Queue Enqueue / Dequeue	$O(1)$	$O(1)$	$O(1)$	Always constant

4. Implementation

4.1 Features built

- **Login and registration** Gmail-only validation enforced across all forms. New users select their city from a dropdown of the seven cities where PostalMS has branches.
- **Parcel sending** Domestic UK city dropdown or international to 24 countries. Price formula: $(ServicePrice + Weight \times 1.2) \times SizeMultiplier \times InternationalMultiplier$.
- **Delivery tracking** Full timeline with driver, route, attempt count and visual progress tracker.
- **Stamps ordering** Browse 6 stamp types, order with delivery or collect, pay online or in store, track orders, missed pickup auto-reschedules after 48 hours.
- **Find Us** 76 locations across 7 UK cities. Auto-detects user city. Google Maps links on every card.
- **Refund requests** Tracking ID, reason dropdown, description and email. Confirmation tells the user to expect a response in 2-3 working days.
- **AI Assistant** Offline keyword-based chat assistant built using rule-based pattern matching (Russell and Norvig, 2021). Handles parcel tracking, pricing, drop-off locations and FAQs without internet.

4.2 Challenges

- **Stamps redesign** The stamps section was rebuilt from scratch as a standalone page after the original design inside the parcels page became too complex to maintain.
- **ProfileView bug** A query was failing on a missing column for some database configurations. Fixed by querying only columns guaranteed to exist.
- **Stamp orders disappearing** MainForm was creating a new StampsView instance on every navigation, wiping the order list. Fixed by caching the instance.
- **Control overlaps** Several forms had overlapping controls due to insufficient layout planning. Required multiple correction passes.

5. Testing

We used MSTest for unit testing. 78 tests across 9 classes, all passing with zero failures in under 600 milliseconds. The test file is fully self-contained with no project reference or database connection needed.

Test Class	Tests	Coverage
HashTableTests	12	Add, search, delete, resize, clear, ContainsKey

Test Class	Tests	Coverage
BSTTests	13	Insert, search, delete, in-order, FindMin, Contains, Clear
QueueTests	10	Enqueue, dequeue, peek, FIFO order, exception handling
PriceCalculationTests	9	Price formula across all service types, sizes and zones
TrackingIDTests	6	Tracking ID format -- prefix, year, zero-padded number
DataStructureIntegrationTests	5	All three structures working together
GmailValidationTests	8	Gmail enforcement across all four forms
StampPriceTests	6	Stamp price with 20% VAT and 2% service fee
CityLocationTests	8	City detection, case-insensitive matching, unknown city fallback

- **Approach** Every test follows the Arrange-Act-Assert pattern. Exception tests use try-catch blocks for compatibility with .NET Framework 4.7.2.
- **Coverage** Tests cover all three data structure operations, the price formula, tracking ID generation, Gmail validation, stamp pricing and city detection logic.

6. Reflection

What went well

- **Data structures** All three implementations worked correctly. 78 tests passed with zero failures on the first run.
- **Teamwork** Clear roles from day one meant no duplicated effort. The four-sprint structure with weekly meetings kept the project on track.
- **Application quality** The app went beyond the minimum requirements with features like the food-app style stamp ordering, city-based location detection, missed pickup rescheduling and the offline AI assistant.

Limitations and what we would do differently

- **UI layout planning** We spent more time than expected fixing overlapping controls. Sketching each form before building it would have saved multiple correction rounds.
- **Self-contained tests from day one** Making the test file depend on a project reference caused build errors early on. Starting self-contained would have been cleaner.
- **Start documentation earlier** The report and supporting documents were left too late. Writing them in parallel with development would reduce end-of-project pressure.
- **Database schema design** Adding the City column to the Users table midway through required updating the stored procedure, sample data and multiple C# files. A more complete schema design upfront would have avoided this.

- **Input file configuration** The database connection string is currently hardcoded in DatabaseHelper. For a production system this should be read from a configuration file so users can specify their own server details.

7. Conclusion

PostalMS is a fully working postal service management system built in C# WinForms with SQL Server. The brief asked for a working application with three custom data structures, unit testing and documentation. We delivered all of that and went further with the stamps ordering system, city-based Find Us, the offline AI assistant and the refund flow.

- **Data structures** Custom Hash Table $O(1)$ lookup, BST $O(\log n)$ sorted display and Queue $O(1)$ FIFO processing -- all built from scratch.
- **Testing** 78 unit tests across 9 classes. All pass with zero failures.
- **Sprints** 4 sprints, 71 tasks, all completed.
- **Documentation** Unit testing report, pseudocode, time complexity analysis, sprint planning and meeting minutes all completed.
- **Extra marks** The offline AI assistant satisfies the extra marks criteria for implementing an AI Agent in the software.

The main limitation is the hardcoded connection string which would need to be made configurable for a real production deployment. Aside from that the application meets all the requirements in the brief.

sReferences

- 1) Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: MIT Press.
- 2) Goodrich, M.T., Tamassia, R. and Goldwasser, M.H. (2013) Data Structures and Algorithms in Java. 6th edn. Hoboken, NJ: Wiley.
- 3) Knuth, D.E. (1998) The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd edn. Reading, MA: Addison-Wesley.
- 4) Microsoft (2024a) Stored Procedures (Database Engine). Available at: <https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures> (Accessed: 10 April 2026).
- 5) Microsoft (2024b) C# documentation. Available at: <https://learn.microsoft.com/en-us/dotnet/csharp> (Accessed: 10 April 2026).

- 6) Microsoft (2024c) MSTest documentation. Available at: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-with-mstest> (Accessed: 10 April 2026).
- 7) Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach. 4th edn. Hoboken, NJ: Pearson.
- 8) Sedgewick, R. and Wayne, K. (2011) Algorithms. 4th edn. Upper Saddle River, NJ: Addison-Wesley.